DOCKER NETWORKING

What is Docker Networking?
Docker Networking is the way Docker allows containers to communicate with each other, communicate with the Docker host, and access external networks such as the internet. Suppose we have an e-commerce application with three containers: Frontend, Backend, and Database. The Frontend container needs to communicate with the Backend container, and the Backend container needs to communicate with the Database container. Docker Networking provides the required network connectivity between these containers. Docker provides different network drivers such as Bridge, Host, None, and Overlay. The most commonly used network for containers running on the same Docker host is the Bridge network. When containers are connected to the same user-defined Bridge network, they can communicate with each other using container names. For example, the Backend container can connect to the Database container using database:3306 instead of manually knowing the Database container IP address. Docker also provides port mapping to allow external users to access an application running inside a container. For example, 8080:80 means traffic coming to port 8080 on the host is forwarded to port 80 inside the container.

EXAMPLE:

Suppose our e-commerce application has:

Frontend Container
↓
Backend Container
↓
Database Container

We create a Docker network:

E-commerce Network
↓
Frontend
Backend
Database

Now the Frontend can communicate with the Backend, and the Backend can communicate with the Database through the Docker network.

For example:

Frontend → backend:8080
Backend → database:3306

The container names can be used for communication when the containers are connected to the same user-defined network.

Simple flow:

User
↓
Host Port
↓
Frontend Container
↓
Docker Network
↓
Backend Container
↓
Docker Network
↓
Database Container

Main point:

Docker Networking = Allows containers to communicate with each other, the host, and external networks.

You said:

exact telugu lo chpu

ChatGPT said:
అవును. ఇక నుంచి Telugu explanation + English technical terms తో, చాలా simple గా చెప్తాను.

Docker Networking అంటే ఏమిటి?
Docker Networking అంటే Docker containers ఒకదానితో ఒకటి ఎలా communicate అవుతాయి, Docker host తో ఎలా communicate అవుతాయి, అలాగే బయట Internet కి ఎలా connect అవుతాయి అనేదే Docker Networking.

ఒక E-commerce application example తీసుకుందాం. మన application లో Frontend Container, Backend Container, Database Container ఉన్నాయి అనుకుందాం. Frontend కి Backend తో మాట్లాడాలి, Backend కి Database తో మాట్లాడాలి. ఈ containers మధ్య communication జరగడానికి Docker Networking ఉపయోగపడుతుంది.

ఉదాహరణకు:

Frontend Container
↓
Backend Container
↓
Database Container

ఈ మూడు containers ని ఒకే Docker Network లో connect చేస్తే, అవి ఒకదానితో ఒకటి communicate చేసుకోవచ్చు.

ఉదాహరణకు Backend container, Database container ని direct IP address ద్వారా కాకుండా container name ద్వారా access చేయవచ్చు.

Backend Container
↓
Database Container
↓
database:3306

ఇక్కడ database అనేది Database container పేరు, 3306 అనేది database service port.

అలాగే బయట ఉన్న user మన application ని access చేయాలంటే port mapping ఉపయోగిస్తాం.

ఉదాహరణకు:

User
↓
Host Port 8080
↓
Container Port 80
↓
Frontend Container

అంటే user host machine లో 8080 port కి request పంపితే, Docker ఆ request ని container లోని 80 port కి forward చేస్తుంది.

Docker లో ముఖ్యమైన network types:

Bridge
Host
None
Overlay

Bridge network సాధారణంగా ఒకే Docker host లో ఉన్న containers మధ్య communication కోసం ఉపయోగిస్తాం.

Host network లో container host machine యొక్క network ని directly use చేస్తుంది.

None network లో container కి normal network connectivity ఉండదు.

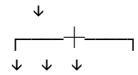
Overlay network multiple Docker hosts లో ఉన్న containers మధ్య communication కోసం ఉపయోగించవచ్చు, ముఖ్యంగా Docker Swarm వంటి environments లో.

1, What are Docker Network Types?

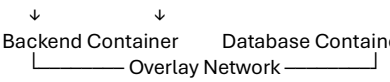
Docker Network Types are different ways Docker provides network connectivity to containers. Docker provides different network drivers such as **Bridge, Host, None, and Overlay**. Each network type is used for a different networking requirement. The **Bridge network** is commonly used when containers are running on the same Docker host and need to communicate with each other. The **Host network** allows a container to use the host machine's network directly. The **None network** provides no normal network connectivity to the container. The **Overlay network** is used to allow containers running on different Docker hosts to communicate with each other, especially in Docker Swarm environments.

EXAMPLE:

Suppose our e-commerce application has:
Frontend Container
Backend Container
Database Container
All three containers are running on the same Docker host.
We can connect them using a Bridge network:



Now the containers can communicate with each other through the Docker network.
For example:
Frontend → backend:8080
Backend → database:3306
Now suppose our application is running on multiple Docker hosts:

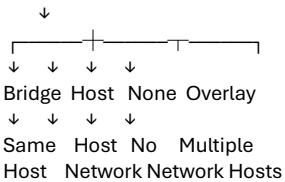


In this situation, an Overlay network can provide communication between containers running on different Docker hosts.

Main Network Types:

Bridge Network → Used for communication between containers on the same Docker host.
Host Network → Container uses the host machine's network directly.
None Network → Container has no normal network connectivity.
Overlay Network → Used for communication between containers running on different Docker hosts.

Simple Flow:



Main point:

Bridge = Same Docker host
Host = Uses host network
None = No normal network
Overlay = Multiple Docker hosts

1. **You said:**

inka

2. **ChatGPT said:**

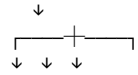
Absolutely bro. 😄 Let's continue in the **same exact style**.

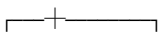
3. What is Docker Bridge Network?

A Docker Bridge Network is a network that allows containers running on the same Docker host to communicate with each other. It is one of the most commonly used Docker network types. When containers are connected to the same user-defined Bridge network, they can communicate with each other using container names instead of manually using container IP addresses. Docker creates an internal network for these containers and manages the communication between them.

EXAMPLE:

Suppose our e-commerce application has:
Frontend Container
Backend Container
Database Container
We create a Docker network:



Frontend Backend Database
Now all three containers are connected to the same Bridge network.
The Frontend can communicate with the Backend:
Frontend → backend:8080
The Backend can communicate with the Database:
Backend → database:3306
Here, backend and database are container names.
We don't need to manually find the container IP address.
For example, instead of:
Backend → 172.18.0.3:3306
we can use:
Backend → database:3306
This is one of the main benefits of using a **user-defined Bridge network**.
We can create a Bridge network using:
docker network create ecommerce-network
Then we can run containers using that network:
docker run -d --name backend --network ecommerce-network backend-image
docker run -d --name database --network ecommerce-network database-image
Now both containers are connected to the same network and can communicate using their container names.
Simple Flow:
Docker Host
↓
E-commerce Bridge Network
↓

Frontend Backend Database
↓
database:3306
Main point:
Docker Bridge Network = Allows containers on the same Docker host to communicate with each other through a private Docker network.

---

4. What is Docker Host Network?
Docker Host Network is a networking mode where a container uses the network stack of the Docker host directly. In this mode, the container does not get a separate network namespace in the same way as a normal Bridge-networked container. The container can use the host's network interfaces and IP address directly.
EXAMPLE:
Suppose our application is running on a Docker host:
Docker Host
IP → 192.168.1.10
We run a container using Host networking:
Docker Host
↓
Host Network
↓
Application Container
The container uses the host's network directly.
Unlike Bridge networking, we don't normally need Docker port mapping such as:
8080:80
because the container is using the host network stack.
For example:
Application Container
↓
Host Network
↓
Host IP
192.168.1.10
Simple Flow:
Application
↓
Container
↓
Host Network
↓
Docker Host
↓
External Network
Main point:
Host Network = Container uses the Docker host's network directly.

---

5. What is Docker None Network?
Docker None Network is a network mode that provides a container with no normal network connectivity. When a container uses the none network, Docker does not connect it to the normal Docker networks or external networks.
EXAMPLE:
Suppose we have a container that performs a task that does not require network access.
Application Container
↓
None
↓
No normal network connectivity
The container is isolated from normal network communication.
For example:
Container
↓
None Network
↓
No Internet
No Container Communication
No External Network
This can be useful when an application does not need network access and we want stronger network isolation.
Simple Flow:
Container
↓
None Network
↓
No Normal Network
Main point:
None Network = Container runs without normal network connectivity.

---

6. What is Docker Overlay Network?
Docker Overlay Network is a network that allows containers running on different Docker hosts to communicate with each other. It is mainly used in distributed Docker environments such as Docker Swarm, where applications can run across multiple Docker hosts.
EXAMPLE:
Suppose our e-commerce application is running on two Docker hosts:
Docker Host 1
↓
Backend Container
and:
Docker Host 2
↓
Database Container
These containers are running on different machines.
We can use an Overlay network:
Docker Host 1 Docker Host 2
↓ ↓
Backend Container Database Container
└──── Overlay Network ────┘
Now the containers can communicate through the Overlay network.
For example:
Backend Container
↓
Overlay Network
↓
Database Container
This is useful when our application is distributed across multiple Docker hosts.
Simple Flow:
Docker Host 1
↓
Backend Container
↓
Overlay Network
↓
Docker Host 2
↓
Database Container
Main point:
Overlay Network = Allows containers running on different Docker hosts to communicate with each other.

---

7. What is Docker Port Mapping?
Docker Port Mapping is the process of mapping a port on the Docker host to a port inside a container. It allows external users or applications to access a service running inside a container.

Containers normally have their own network namespace, so simply running an application inside a container does not automatically make it accessible through the host's network interface. Port mapping allows us to expose the required container port through the Docker host.

EXAMPLE:

Suppose our Frontend application is running inside the container on port 80.

Frontend Container

Port → 80

We want users to access it through port 8080 on the Docker host.

We can use:

8080:80

This means:

Host Port 8080

↓

Container Port 80

For example:

docker run -d -p 8080:80 frontend-image

Now when a user accesses:

Docker Host:8080

Docker forwards the traffic to:

Container:80

Simple Flow:

User

↓

Docker Host

↓

Port 8080

↓

Container Port 80

↓

Frontend Application

Main point:

1. **Port Mapping = Maps a host port to a container port so external traffic can reach the application running inside the container.**

_-

ChatGPT said:

Yes bro 👍 Already covered topics repeat cheyyakunda, **pending important Docker Networking topics** matrame same style lo continue chestha.

8. What is the difference between Default Bridge and User-defined Bridge Network?

Docker provides a default Bridge network called bridge. When we create containers without specifying a network, they are normally connected to this default Bridge network. Docker also allows us to create our own user-defined Bridge networks. User-defined Bridge networks provide better container-to-container communication because containers can communicate with each other using container names through Docker's built-in DNS. They also provide better isolation and allow us to manage which containers belong to the same network.

EXAMPLE:

Suppose we have:

Frontend Container

Backend Container

Database Container

If we use a user-defined network:

E-commerce Network

↓

Frontend

Backend

Database

Now Backend can communicate with Database using:

database:3306

Instead of manually using the Database IP address.

We can create a user-defined Bridge network using:

docker network create ecommerce-network

Then connect containers to it:

docker run -d --name backend --network ecommerce-network backend-image

docker run -d --name database --network ecommerce-network database-image

Main point:

Default Bridge = Docker's default network.

User-defined Bridge = Custom network with better container communication, DNS, and isolation.

---

9. What is Docker DNS?

Docker DNS is the mechanism that allows containers connected to the same user-defined network to find and communicate with each other using container names instead of IP addresses.
Container IP addresses can change when containers are recreated. So depending directly on IP addresses is not a good approach.
Docker provides internal DNS so that a container can use another container's name to communicate with it.
EXAMPLE:
Suppose we have:
Backend Container
Database Container
Both are connected to:
E-commerce Network
Instead of:
Backend → 172.18.0.5:3306
Backend can use:
Backend → database:3306
Here:
database
↓
Docker DNS
↓
Database Container IP
If the Database container is recreated and its IP changes, Docker DNS can resolve the database name to the new IP.
Simple Flow:
Backend
↓
database:3306
↓
Docker DNS
↓
Database Container
Main point:
Docker DNS = Allows containers to discover and communicate with each other using names instead of manually managing IP addresses.

---

10. What is Docker EXPOSE?
EXPOSE is a Dockerfile instruction that documents which port an application inside the container is intended to listen on.
It does **not** publish the port to the Docker host by itself.
EXAMPLE:
Suppose our application runs on port 8080.
In the Dockerfile:
EXPOSE 8080
This tells users and Docker-related tooling that the application is expected to listen on port 8080.
But this does not mean users can automatically access it through the host.
To publish the port, we need port mapping:
docker run -d -p 8080:8080 my-app
Now:
Host Port 8080
↓
Container Port 8080
Main point:
EXPOSE = Documents the intended container port.
-p **= Actually publishes/maps the port to the host.**

---

11. EXPOSE vs Port Mapping
EXPOSE and port mapping are different concepts. EXPOSE is mainly documentation about the port the application listens on inside the container, while port mapping using -p makes the container port accessible through a port on the Docker host.
EXAMPLE:
Dockerfile:
EXPOSE 80
This means:
Application
↓
Container Port 80
But external users cannot automatically access it through the host just because EXPOSE 80 exists.
If we run:
docker run -d -p 8080:80 my-app
then:
User
↓
Host Port 8080
↓
Container Port 80

↓
Application
Simple comparison:
EXPOSE 80
↓
Documents container port

-p 8080:80
↓
Publishes host port 8080
to container port 80
Main point:
EXPOSE = Documentation
-p **= Port publishing / mapping**

---

12. How do containers communicate with each other?
Containers communicate with each other through a Docker network. When containers are connected to the same user-defined network, they can communicate using their container names.
EXAMPLE:
Suppose our application has:
Frontend
Backend
Database
We connect all three to:
E-commerce Network
Now:
Frontend
↓
backend:8080
↓
Backend

Backend
↓
database:3306
↓
Database
The containers do not need to know each other's IP addresses manually.
Simple Flow:
Frontend
↓
Docker Network
↓
Backend
↓
Docker Network
↓
Database
Main point:
Containers on the same Docker network can communicate with each other using container names.

---

13. How do you create a Docker Network?
We can create a Docker network using the docker network create command.
EXAMPLE:
Suppose we want to create a network for our E-commerce application.
docker network create ecommerce-network
Now Docker creates:
E-commerce Network
We can run a container using this network:
docker run -d --name backend --network ecommerce-network backend-image
Another container:
docker run -d --name database --network ecommerce-network database-image
Now:
Backend
↕
E-commerce Network
↕
Database
They can communicate with each other.
Main point:
docker network create **= Creates a Docker network.**

14. What are important Docker Network commands?
Docker provides commands to create, inspect, connect, disconnect, and remove networks.
EXAMPLE:
To list networks:
docker network ls
To create a network:
docker network create ecommerce-network
To inspect a network:
docker network inspect ecommerce-network
To connect a running container to a network:
docker network connect ecommerce-network backend
To disconnect a container:
docker network disconnect ecommerce-network backend
To remove a network:
docker network rm ecommerce-network
Main point:
The important commands are:
docker network ls
docker network create
docker network inspect
docker network connect
docker network disconnect
docker network rm

---

15. What is Container IP Address?
Every container connected to a Docker network can receive an IP address from that network. This IP address can be used for network communication, but applications should generally avoid depending on a container's IP because the IP can change when the container is recreated.
EXAMPLE:
Suppose our Database container gets:
Database
IP → 172.18.0.5
Backend can technically communicate using:
172.18.0.5:3306
But if the Database container is recreated:
Old IP → 172.18.0.5
New IP → 172.18.0.7
The IP changed.
Instead, with a user-defined network, we can use:
database:3306
Docker DNS resolves the name to the current container IP.
Main point:
Container IP = Can change when the container is recreated, so prefer container/service names for application communication.

---

16. What is Network Isolation in Docker?
Docker Network Isolation means we can control which containers can communicate with each other by placing them on different networks.
EXAMPLE:
Suppose our application has:
Frontend
Backend
Database
We can create separate networks:
Frontend Network
↓
Frontend
Backend
and:
Backend Network
↓
Backend
Database
Now the Backend can communicate with both networks, while the Database does not need to be directly connected to the Frontend network.
Frontend→ Frontend Network→ Backend→ Backend Network→ Database
This provides better network organization and isolation.
Main point:
Network Isolation = Controls which containers can communicate with each other by separating them into different networks.